HIDRO SMART · ENTREGA DE PROYECTO FORMATIVO

Implementación y operación

Documento de trabajo para revisión del equipo

Borrador de revisión · no es versión aprobada

Corte documental: 2026-11-04
Fuente funcional acordada: PT_HS · hu-20-dev · 1f02b78
Carpeta: 06-implementation-procedure

> **Estado y alcance.** Documento en revisión para la entrega. Las fechas, cifras y funciones heredadas del documento fuente solo se consideran vigentes cuando están respaldadas por evidencia del monorepo PT_HS. Consulta el índice de entrega y el estado del borrador.

HIDROSMART

Procedimiento de implementación y operación de HidroSmart

Ejecución local reproducible y controles de puesta en marcha

Versión documental consolidada 1.0
Fecha de corte: 2026-10-09
Código documental: DOC-06

1. Precondiciones

Clonar o disponer del repositorio PT_HS sin modificar la rama fuente.

Instalar Docker/Docker Compose; Node/npm y PlatformIO son necesarios para tareas fuera de contenedores.

Disponer de variables de entorno a partir de los ejemplos del repositorio; nunca copiar secretos reales a Git.

Para pruebas físicas: ESP32, sensor, alimentación, red Wi-Fi y development build móvil si se usa BLE.

2. Flujo de implementación local

Paso	Acción	Resultado esperado
1	Revisar rama, README, .env.example y puertos.	Ambiente identificado sin secretos expuestos.
2	Preparar variables para BD, backend, frontend y MQTT.	Servicios pueden resolver nombres y credenciales locales.
3	Levantar PostgreSQL, broker, backend y frontend mediante docker compose.	Servicios en ejecución; verificar logs.
4	Ejecutar db-bootstrap/Liquibase conforme al compose.	Changelog actualizado y BD lista.
5	Comprobar API /api/v1 y frontend en puerto 5173.	HTTP y aplicación responden.
6	Publicar/recibir un mensaje MQTT de prueba controlado.	Ingestión valida y registra o rechaza de forma trazable.
7	Construir/cargar firmware PlatformIO y aprovisionar BLE.	ESP32 conecta a Wi-Fi/MQTT y publica estado.
8	Ejecutar pruebas del DOC-07 y guardar evidencia fechada.	Criterios aceptados o incidencias abiertas.

3. Orden recomendado de verificación

Contenedores y puertos.

Liquibase status/validate y estado de migraciones.

Conectividad del backend con BD.

Conectividad del backend con broker.

Rutas de autenticación y usuario de prueba.

Ingreso MQTT con mensaje válido, inválido y duplicado.

Consulta de lectura desde frontend.

Firmware y prueba física, si aplica.

4. Firmware y aprovisionamiento

El firmware se gestiona con PlatformIO, entorno esp32dev, framework Arduino y monitor a 115200. El flujo BLE guarda SSID, contraseña, host, puerto y deviceCode en NVS; puede conservar hasta cinco perfiles. El hardwareId se deriva del eFuse y puede actuar como código por defecto. El móvil requiere development build para react-native-ble-plx.

5. Detención, actualización y rollback

Antes de actualizar migraciones o imágenes, registrar commit, fecha, variables y estado de la BD.

Tomar backup y comprobar restauración antes de cambios destructivos.

Aplicar migraciones de forma ordenada; no editar manualmente la base para “hacer coincidir” diagramas.

Si una verificación falla, detener la liberación, conservar logs y volver a la versión previamente validada según el procedimiento del entorno.

No existe en el material revisado una prueba completa de backup/restore; queda como requisito de cierre.

6. Producción: controles obligatorios

Control	Estado en corte	Acción antes de producción
MQTT TLS/ACL	El firmware tiene perfil externo Flespi/TLS; Mosquitto local sigue sin TLS para desarrollo. ACL y token del entorno deben verificarse.	Verificar ACL de mínimo privilegio, token individual por placa y token separado para backend; no exponer Mosquitto local.
Secretos	Variables previstas; valores reales no documentados.	Usar gestor seguro y rotación.
Backups	No verificado end-to-end.	Probar backup, restore y RPO/RTO.
Correo transaccional Brevo	Integración por API HTTPS documentada; estado de variables privadas y entrega real no se deduce del repositorio.	Probar cada flujo conectado y confirmar aceptación/entrega en Brevo; no asumir alertas por correo sin disparador.
Observabilidad	Logs disponibles; SLA no fijado.	Definir métricas, alarmas y retención.

Nota de control documental. Este procedimiento describe una ruta operativa basada en docker-compose.yml, README y docs del repositorio. Los comandos exactos pueden variar con el entorno; el operador debe revisar los scripts vigentes antes de ejecutar acciones sobre datos.

Despliegue

Despliegue y operación

Arquitectura de ambientes

Capa	Servicio previsto	Configuración principal
Web	Cloudflare Pages	Compilar `FT_HS/Web`; salida `dist`; `VITE_API_URL` apunta a la API HTTPS.
API	Render	Ejecutar `BK_HS`; configurar CORS con el origen exacto de Pages.
PostgreSQL	Neon	`DATABASE_URL` para el pool principal y `DB_INGEST_DATABASE_URL` cuando se separe el usuario de ingesta. Requerir TLS del proveedor.
MQTT	Flespi	`mqtts://mqtt.flespi.io:8883`; token ACL como usuario MQTT; contraseña vacía.
Correo	Brevo	`BREVO_API_KEY`, remitente verificado y nombre de remitente.

Esta tabla describe la arquitectura objetivo de PT_HS. No sustituye la comprobación del commit ni de las variables guardadas en cada proveedor.

URLs públicas indicadas para la entrega

Servicio	URL	Nota
Aplicación web, inicio de sesión	https://pt-hs.pages.dev/login	URL proporcionada por el equipo. Verificar disponibilidad y rutas principales antes de grabar evidencias.
Sitio base	https://pt-hs.pages.dev	Dominio de Cloudflare Pages indicado para el frontend.

La inclusión de estas URLs en la documentación no constituye una prueba de disponibilidad del despliegue.

Variables

Los nombres de las variables se documentan; sus valores secretos se cargan desde el administrador de secretos del proveedor y nunca se guardan en Git.

Render (backend)

- `NODE_ENV=production`, `PORT`, `CORS_ORIGIN`, `FRONTEND_URL` y `PASSWORD_RESET_URL`.
- `DATABASE_URL` o `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`.
- `DB_INGEST_DATABASE_URL` o `DB_INGEST_USER`, `DB_INGEST_PASSWORD` y `DB_INGEST_POOL_MAX`.
- `JWT_SECRET`, `JWT_EXPIRES_IN`, `JWT_REFRESH_EXPIRES_IN`, `JWT_ISSUER` y `JWT_AUDIENCE`.
- `BREVO_API_KEY`, `BREVO_FROM_EMAIL` y `BREVO_FROM_NAME`.
- `MQTT_BROKER_URL`, `MQTT_CLIENT_ID`, `MQTT_USERNAME`, `MQTT_PASSWORD` y `MQTT_QOS`.

Para el sitio actual, incluye `https://pt-hs.pages.dev` en `CORS_ORIGIN` y úsalo como `FRONTEND_URL`. Configura `PASSWORD_RESET_URL=https://pt-hs.pages.dev/login` para que los enlaces de recuperación vuelvan a la web publicada. `CORS_ORIGIN` acepta una lista separada por comas si se permiten otros orígenes.

Para Flespi usa una URL TLS y un token ACL con permisos mínimos. El token del backend debe poder suscribirse a telemetría/estado de dispositivos y publicar únicamente configuración/comandos autorizados. Cada ESP32 debe tener su propio token ACL, limitado a publicar los topics de su dispositivo. No reutilices el token maestro ni el del backend en el firmware.

Cloudflare Pages (web)

- Root directory: `FT_HS/Web` si se conecta el repositorio PT_HS.
- Build command: `npm run build`.
- Build output: `dist`.
- Node.js: `22`.
- `VITE_API_URL=https://pt-hs.onrender.com/api/v1` para producción. Usa la URL HTTPS de la API del ambiente correspondiente en previews.

Define `VITE_API_URL` tanto en Production como en Preview. El `vite.config.js` detiene el build de producción si falta esta variable y exige HTTPS en Cloudflare Pages; así evita generar un bundle que intente llamar a `localhost` o a una API insegura.

`VITE_*` se incorpora al bundle del navegador: nunca pongas tokens, contraseñas ni claves privadas en estas variables.

Cloudflare Pages sirve las rutas de una SPA desde `index.html` si no se publica un `404.html` de nivel superior. Mantén la salida de Vite sin un `404.html` estático para que `/login` y las demás rutas del frontend abran directamente.

Promoción

1. Seleccionar el commit y la rama que aprobaron la revisión y CI.
2. Verificar que la migración Liquibase esté incluida y que el backup de la base esté vigente.
3. Aplicar migraciones en el ambiente destino con credenciales de despliegue restringidas.
4. Desplegar backend y verificar `/health` y logs de arranque.
5. Desplegar Pages y comprobar login, hogar, consumo, alertas y correo en el ambiente.
6. Con credenciales MQTT seguras, confirmar conexión TLS y suscripción del backend sin publicar datos de prueba en topics productivos.

Desarrollo local

Desde la raíz de PT_HS, copia `.env.example` a `.env`, reemplaza los valores de ejemplo y no compartas ese archivo:

```
Copy-Item .env.example .env
docker compose --env-file .env up -d postgres db-bootstrap mosquitto
docker compose --env-file .env --profile tooling run --rm liquibase validate
docker compose --env-file .env --profile tooling run --rm liquibase update
docker compose --env-file .env up -d --build backend frontend
```

Comprueba `http://localhost:3000/health` y `http://localhost:5173/health`. Mosquitto anónimo sin TLS se limita al desarrollo local y no debe exponerse a Internet ni a una red compartida.

Reversión

Si una promoción falla, detén la promoción posterior, conserva logs y el identificador del commit, revierte la aplicación al último artefacto estable y usa el rollback Liquibase solo después de evaluar compatibilidad/datos. No restaures una base sobre producción sin aprobación operativa y backup verificado.

Instalación y dependencias

Instalación de dependencias en Windows

Esta guía prepara un equipo nuevo para desarrollar HidroSmart. El proyecto se divide en tres alcances: la aplicación web y la pila Docker, el firmware del ESP32 y la aplicación Android. Instala primero lo obligatorio y añade lo opcional solo cuando vayas a trabajar esa parte.

> HidroSmart usa Node.js 22 en sus imágenes Docker. El frontend móvil usa Expo SDK 57 y React Native 0.86.3; verifica las versiones exactas en `FT_HS/frontend/package.json` y no actualices Expo globalmente.

1. Descargas obligatorias para la aplicación web

Herramienta	Versión o criterio	Para qué se usa	Descargar
Git	Versión estable actual	Clonar, actualizar y subir el repositorio.	git-scm.com
Docker Desktop	Versión estable actual, con WSL 2 habilitado	Ejecutar PostgreSQL, Mosquitto, Liquibase, backend y frontend.	Docker Desktop
Node.js	**22 LTS**	Ejecutar las pruebas, herramientas del backend, Vite y Expo.	nodejs.org
Navegador Chromium	Edge o Chrome actual	Usar la web y Web Bluetooth durante el aprovisionamiento.	Edge o Chrome

Visual Studio Code es recomendado para editar el proyecto, pero no es un requisito de ejecución. Se puede descargar desde code.visualstudio.com.

Lo que no debes instalar por separado

Docker Compose descarga y ejecuta estas dependencias con las versiones fijadas en `docker-compose.yml`:

- PostgreSQL 16.
- Eclipse Mosquitto 2.0.21.
- Liquibase y el controlador PostgreSQL para aplicar migraciones.
- Nginx para servir el frontend construido.

Tampoco instales Express, React, Vite, Expo, ArduinoJson, PubSubClient o NimBLE manualmente. Los gestores del proyecto resuelven sus versiones desde los archivos `package-lock.json` y `platformio.ini`.

2. Verificar las herramientas base

Abre una terminal nueva de PowerShell y ejecuta:

```
git --version
docker --version
docker compose version
node --version
npm.cmd --version
```

`node --version` debe mostrar `v22.x`. En este proyecto se usa `npm.cmd` en PowerShell para evitar el bloqueo de ejecución de `npm.ps1` que puede aplicar Windows a los scripts.

Si Docker Desktop solicita WSL 2 o virtualización, completa ese asistente y reinicia Windows antes de continuar. No es necesario instalar PostgreSQL local ni configurar un servidor MQTT de Windows.

3. Descargar dependencias del repositorio

Desde la raíz `PT_HS`, crea la configuración local y descarga las dependencias JavaScript exactas de cada aplicación:

```
Copy-Item .env.example .env
npm.cmd --prefix BK_HS ci
npm.cmd --prefix FT_HS/frontend ci
```

`npm ci` usa las versiones bloqueadas del repositorio y recrea `node_modules`. Úsalo para instalar o recuperar un entorno; usa `npm.cmd install` solamente cuando modifiques deliberadamente un `package.json` y su lockfile.

No ejecutes `npm audit fix --force`: puede cambiar Expo, React Native o Vite de forma incompatible. Las actualizaciones de dependencias se revisan y prueban por separado.

4. Levantar la plataforma local

Con Docker Desktop iniciado, ejecuta desde `PT_HS`:

```
docker compose --env-file .env config --quiet
docker compose --env-file .env up -d postgres db-bootstrap mosquitto
docker compose --env-file .env --profile tooling run --rm liquibase validate
docker compose --env-file .env --profile tooling run --rm liquibase update
docker compose --env-file .env up -d --build backend frontend
docker compose --env-file .env ps
```

Luego comprueba:

```
Invoke-WebRequest http://localhost:3000/health
Invoke-WebRequest http://localhost:5173/health
```

La aplicación queda disponible en `http://localhost:5173`. La API se publica solo en `localhost:3000`, PostgreSQL en `localhost:5433` y MQTT en el puerto `1883` de la red local configurada para desarrollo.

Antes de conectar un ESP32 real, cambia las contraseñas de ejemplo en `.env` y define `MQTT_DEVICE_HOST` con un nombre DNS o reserva DHCP que el ESP32 pueda resolver desde la Wi-Fi. Nunca uses `localhost` ni el nombre interno `mosquitto` como host del dispositivo físico.

5. Solo si vas a compilar o cargar el ESP32

Descargas adicionales

Herramienta o material	Cuándo hace falta
PlatformIO IDE para VS Code **o** PlatformIO Core	Siempre que compiles, monitorees o cargues firmware. Elige una de las dos opciones; la extensión es la más cómoda.
Controlador USB CP210x o CH340	Solo si Windows no crea un puerto `COM` al conectar la placa. Identifica primero el chip USB de tu placa; no instales controladores al azar.
Cable USB de datos	Siempre. Algunos cables solo cargan y no crean un puerto serie.
ESP32 Dev Module y caudalímetro YF-S201	Para la prueba física del firmware actual.

PlatformIO descarga automáticamente la plataforma `espressif32`, el framework Arduino y las bibliotecas declaradas (`PubSubClient`, `ArduinoJson` y `NimBLE-Arduino`) al ejecutar la compilación.

Primera compilación

```
Set-Location firmware
Copy-Item provisioning\config.example.h provisioning\config.h
pio run
```

```
pio device list
pio run -t upload --upload-port COM6
pio device monitor --baud 115200
```

Sustituye `COM6` por el puerto mostrado por `pio device list`. `config.h` queda ignorado por Git: contiene Wi-Fi y, si corresponde, las credenciales MQTT de la placa. No lo confirmes ni lo envíes por chat.

Si aparece `PermissionError(13)` o "port is busy", cierra el monitor serie, Arduino IDE y cualquier programa que esté usando ese puerto. La guía funcional del dispositivo está en IoT y el contrato de telemetría en firmware/mqtt/contract.md.

6. Solo si vas a trabajar Android/móvil

El móvil sigue en estabilización; prepara primero la web y la pila Docker. Para ver las pantallas en un teléfono Android con Expo Go no necesitas Android Studio: instala Expo Go compatible con SDK 57 y, desde `FT_HS/frontend`, usa `npm.cmd run mobile:start`. El PC y el teléfono deben estar en la misma red; el script prepara el bundle con la IP LAN y muestra el QR. Comprueba desde el teléfono `http://IP_DEL_PC:5173/health` antes de escanear.

Expo Go no contiene `react-native-ble-plx`. Solo para probar aprovisionamiento BLE o compilar un APK propio hace falta un *development build* nativo y las herramientas siguientes.

Descargas adicionales

1. Instala Android Studio y acepta el asistente de instalación estándar.
2. Instala **JDK 17**. Expo documenta JDK 17 para Android en Windows; Android Studio suele traer su propio entorno, pero el JDK evita fallos de Gradle en terminal.
3. En **Settings > Languages & Frameworks > Android SDK**, instala: - Android SDK Platform **36** y sus fuentes. - Android SDK Build-Tools. - Android SDK Platform-Tools (`adb`). - Android Emulator. - Android SDK Command-line Tools (latest).
4. En Device Manager crea y arranca un emulador x86_64 con Google APIs. La virtualización VT-x/AMD-V debe estar activa en BIOS/UEFI.

Android Studio suele instalar el SDK en `%LOCALAPPDATA%\Android\Sdk`. Crea la variable de usuario `ANDROID_HOME` con esa ruta y añade estas rutas a `Path`:

```
%ANDROID_HOME%\platform-tools
%ANDROID_HOME%\emulator
%ANDROID_HOME%\cmdline-tools\latest\bin
```

Cierra y abre PowerShell; después verifica el acceso a ADB:

```
adb --version
adb devices
```

Si `adb` no se reconoce, el SDK está instalado pero `platform-tools` no está en `Path`. Corrige la variable antes de intentar compilar.

Preparar y ejecutar Android

Con el emulador encendido:

```
Set-Location FT_HS/frontend
npm.cmd run mobile:doctor
npm.cmd run mobile:android
```

El emulador puede comunicarse con servicios locales mediante `adb reverse`, que ejecutan los scripts del proyecto. Un teléfono físico no puede usar `localhost`: deberá recibir una URL HTTPS o una dirección/nombre accesible en la LAN para la API.

Antes de modificar React Native, React o paquetes `expo-*`, consulta el `package.json` y ejecuta `npm.cmd run mobile:doctor`; conserva la compatibilidad con Expo SDK 57 mientras se use Expo Go para ese SDK.

7. Validación después de instalar

Ejecuta estas comprobaciones desde `PT_HS`:

```
npm.cmd --prefix BK_HS run check
npm.cmd --prefix BK_HS test
npm.cmd --prefix FT_HS/frontend run lint
npm.cmd --prefix FT_HS/frontend run typecheck
npm.cmd --prefix FT_HS/frontend test
npm.cmd --prefix FT_HS/frontend run build
npm.cmd --prefix FT_HS/frontend run mobile:doctor
```

Para los comandos diarios, diagnóstico de contenedores y pruebas de integración consulta Operación local y Pruebas.